RAGApp Hybrid Architecture

`DRKRAG · NOTEBOOKS/RAGAPP_HYBRID.IPYNB`

Hybrid retrieval over five neurology textbooks, answered by one local model and scored by another. Nothing in this pipeline costs money, and nothing can hit a quota — which is the constraint that shapes every decision below.

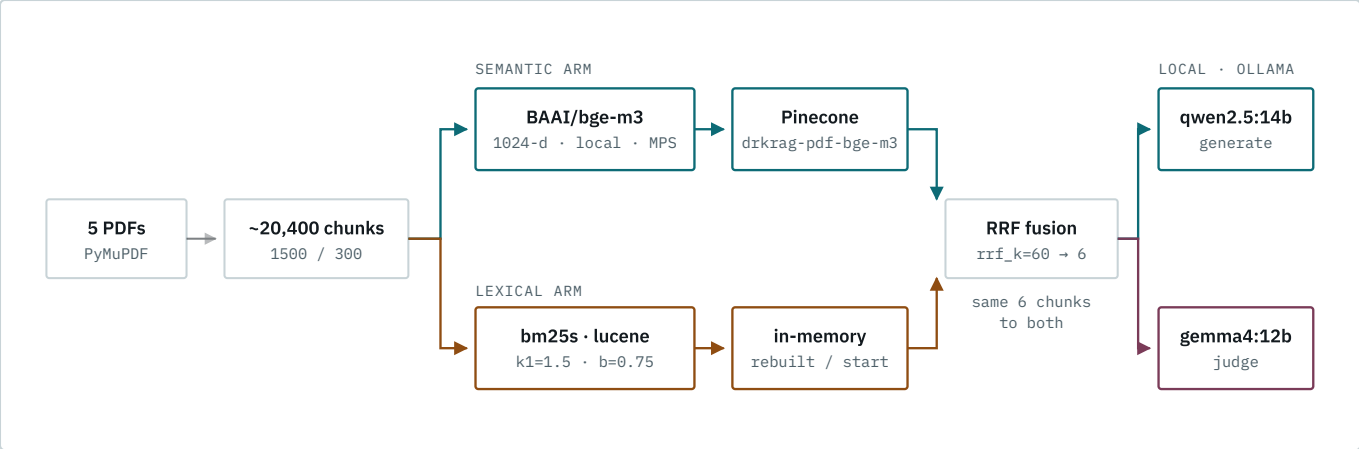
CORPUS	EMBEDDING	INDEXES	GENERATOR	JUDGE	EVAL SET
5 neurology PDFs	`BAAI/bge-m3` · 1024-d	2 — Pinecone + bm25s	1 — `qwen2.5:14b`	1 — `gemma4:12b`	14 — 10 answerable + 4 not

Contents

The stack, end to end

One corpus feeds two independent indexes. The vector arm matches meaning; the lexical arm matches literal terms. Rare clinical tokens — `tofersen`, `SOD1`, `El Escorial`, `physostigmine` — are exactly where embeddings blur and BM25 is exact, so fusion reads both.

The semantic arm runs `BAAI/bge-m3` (568M parameters, 1024-d), replacing `all-MiniLM-L6-v2` (22M parameters, 384-d). MiniLM's weakness on this corpus was measurable: the smoke query `tofersen SOD1` topped out at cosine 0.284 — near-nothing in embedding space — while BM25 returned confident literal matches. BGE-M3 is trained for multilingual and long-context retrieval and holds domain vocabulary far better.



Fusion works on **rank**, not raw score: cosine similarity is bounded 0–1 while BM25 is unbounded and corpus-relative, so the two numbers are not on a comparable scale. A chunk found by both arms outranks one topping only a single arm. The judge receives the same six chunks as the generator — that is what makes `groundedness` and `retrieval_relevance` answerable.

Chunking for this corpus

1500 characters with 300 overlap (~375 tokens, 20% overlap). The size is not set by the embedding model's ceiling — BGE-M3 accepts 8192 tokens, and using anything close to that would be a mistake. Four constraints set it instead.

PRESSURE	DIRECTION	WHY
Clinical units are multi-paragraph	**larger**	Management protocols, diagnostic criteria, and dosing tables run 300–500 tokens. At 250 tokens an ALS management list or the McDonald criteria splits across two chunks, and neither half retrieves well.
Dense vectors dilute	**smaller**	One 1024-d vector averaging thousands of tokens loses the specific fact. BGE-M3's long-context strength is whole-document ranking; for pinpoint retrieval its useful band is roughly 256–512 tokens.
The BM25 arm shares these chunks	**larger**	Co-occurring rare terms (`tofersen` + `SOD1`) landing in one chunk is exactly the hybrid win. BM25's `b=0.75` length normalization absorbs the extra length.
Judge cost scales with chunk size	**smaller**	`groundedness` and `retrieval_relevance` read all six retrieved chunks. Doubling chunk size doubles gemma4:12b's input on the slowest stage in the pipeline.

1500/300 sits where those four balance. Move to 2000/400 only if answers begin truncating mid-protocol; do not go beyond it.

Knock-on effects of the change:

- Chunk count drops from a measured 30,603 at 1000/200 to roughly 20,400 at 1500/300 — an estimate from the stride ratio, not a measured figure.
- Every chunk gets new text, so every content-hash id changes. The whole corpus must be re-embedded; no incremental upsert can absorb this.
- Retrieval `k=6` now carries ~2,250 tokens of context instead of ~1,500. Well inside qwen2.5:14b's window, and the three-sentence answer cap is unchanged.

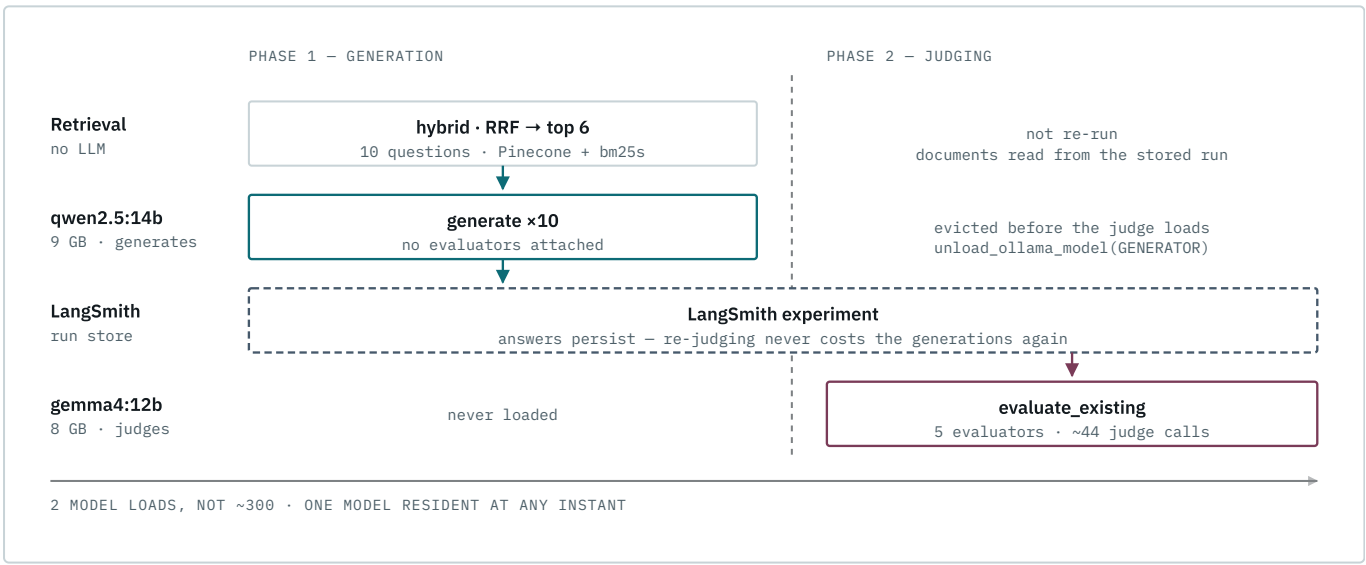
- Scores are **not** comparable to any run made at 1000/200. Treat the first BGE-M3 run as a new baseline rather than a delta.

> **Do both changes at once.** Switching to BGE-M3 already forces a full re-embed into a fresh 1024-d index. Changing chunk size in the same pass costs one re-embed instead of two — tens of minutes each on a 568M-parameter model.

Size is the coarse control. The finer one is **separator choice**: `RecursiveCharacterTextSplitter` defaults to `["\n\n", "\n", " ", ""]`, which knows nothing about headings, numbered protocol steps, or table rows. Tuning separators for the textbooks' structure buys more retrieval quality per unit of effort than further size adjustment, and is the natural next step.

Evaluation runs in two phases

The machine has 19.3 GB of RAM. `qwen2.5:14b` is 9 GB and `gemma4:12b` is 8 GB, so the two cannot both be resident. An interleaved loop — generate, judge ×4, generate — would evict and reload an 8–9 GB model hundreds of times. An earlier version that did exactly that ran for 15.5 hours, finished one mode of three, and recorded zero scores.



The diagonal is the whole point: `qwen2.5:14b` occupies memory in phase 1, `gemma4:12b` in phase 2, and the explicit `keep_alive=0` eviction between them means the two never contend. Two model loads instead of roughly 300.

`RETRIEVAL_MODES` defaults to `["hybrid"]`; set it to all three modes for a retriever comparison at 3× the runtime, with generator and judge still held fixed.

Splitting the phases buys a second thing beyond speed. Phase 1 results persist in LangSmith, so a judge failure — or a decision to change the judge model — never costs you the generations again.

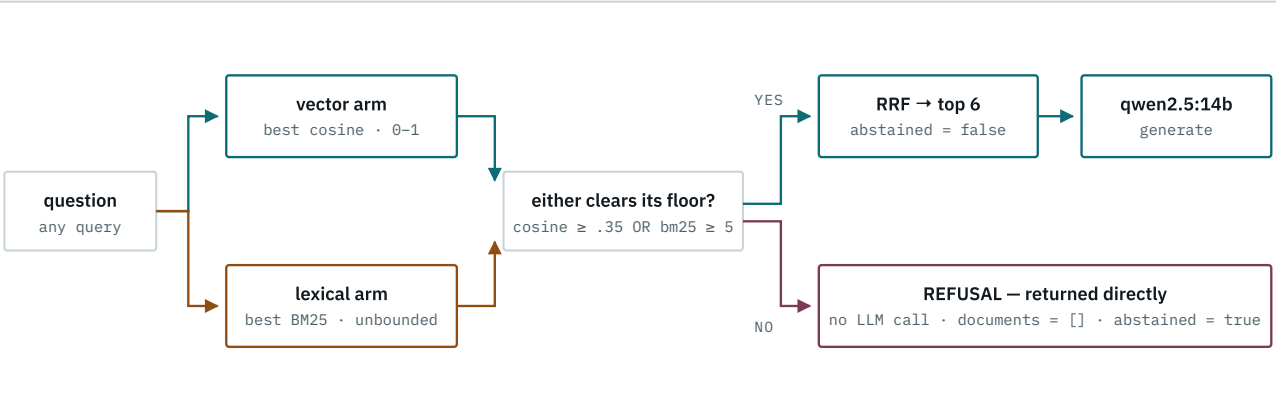
What each piece is

STAGE	TECHNOLOGY	SETTINGS THAT MATTER
Parse	PyMuPDF	Chosen over pypdf, which trips a decompression guard on these textbooks
Chunk	RecursiveCharacterTextSplitter	`chunk_size=1500`, `chunk_overlap=300` → ~20,400 chunks (est.)
Embed	sentence-transformers · `BAAI/bge-m3`	1024-d, normalized, `encode_batch_size=8`, no query/passage prefixes, local on Apple MPS
Vector index	Pinecone serverless	`drkrag-pdf-bge-m3`, dim 1024, cosine, AWS `us-east-1` (free tier is pinned there), SHA-1 content ids
Keyword index	bm25s, lucene variant	`k1=1.5`, `b=0.75`, English stopwords, rebuilt per kernel start
Fusion	Reciprocal Rank Fusion	`rrf_k=60`, `candidate_k=20` per arm, returns top 6
Generate	Ollama · qwen2.5:14b	temperature 0, three-sentence answer cap, 9 GB resident during phase 1
Judge	Ollama · gemma4:12b	`method="json_schema"`, `include_raw=True`, explanation before verdict
Abstention floor	`clears_floor()` in `retrieve()`	`MIN_COSINE=0.35` **or** `MIN_BM25=5.0` — either arm clearing it is enough
Record	LangSmith	Dataset of 10 examples; every `retrieve()` call traced as its own span

> **Reading the scores.** Each metric is the fraction of 10 questions the judge marked true, so resolution is 0.1. A single flipped question moves any metric a full step — treat gaps under 0.2 as noise. Even a 0.2 gap is weak at n=10; check *which* two questions flipped and whether they share a cause.

The refusal path

A RAG app that hallucinates when retrieval fails is worse than one that says it could not find the answer. `retrieve()` can now return nothing, and the bot refuses when it does.



Why the floor reads arm scores, not the fusion score

RRF is built from **rank alone**. The top chunk scores `1/(60+1) + 1/(60+1) ≈ 0.0328` whether it is a perfect match or unrelated boilerplate, so the fused score carries no relevance information and cannot be thresholded. The gate reads the underlying arm scores instead:

SIGNAL	RANGE	COMPARABLE ACROSS QUERIES?
Cosine similarity (vector arm)	0–1, bounded	Yes — a threshold transfers
BM25 score (lexical arm)	unbounded, corpus-relative	No — must be calibrated on this corpus
RRF fusion score	~0.008–0.033, rank-derived	**Carries no relevance signal at all**

Why *either* arm clears it, not both

BM25 exists in this pipeline precisely to catch rare clinical tokens the embedding blurs — `tofersen`, `SOD1`, `El Escorial`. Requiring the vector arm to agree would abstain on exactly the queries BM25 was added for. So the rule is: **abstain only when both arms are weak.**

Refusing without calling the model

When the floor is not cleared, `rag_bot` returns the `REFUSAL` constant directly and never invokes the LLM. Three reasons: the refusal is deterministic, it costs no inference, and it cannot be talked out of by a persuasive-looking near-miss chunk. Sending an empty `Documents:` block instead would read to the model as "answer from your own knowledge" — the hallucination this path exists to prevent.

Calibration is required, not optional

`MIN_COSINE=0.35` and `MIN_BM25=5.0` are **starting points, not measurements**. The notebook has a calibration cell that prints both arm scores for four known in-corpus questions and four known out-of-corpus ones, then reports whether the two groups separate. If they overlap, the floor cannot distinguish them and the thresholds are not trustworthy. Re-run it whenever the embedding model or the chunk size changes — both move the cosine distribution, and BM25 is corpus-relative by construction.

Measuring it

Four unanswerable questions were added to the dataset, each outside a neurology corpus in a different way — a general-knowledge fact, a software question, a private business figure, an unrelated technical domain — so no single quirk of the floor passes all four. Their reference answer *is* the refusal, so `correctness` rewards declining and penalises a confident invention.

The new `abstention` evaluator is a deterministic label comparison — no LLM, no cost, and it reads in both directions:

QUESTION	BOT	VERDICT
unanswerable	abstained	**True** — correctly declined
unanswerable	answered	**False** — hallucinated; the failure that matters
answerable	answered	**True** — correctly proceeded
answerable	abstained	**False** — over-refusal; the floor is too high

That last row is why the metric is not simply "refusal rate": a floor set high enough to refuse everything would score perfectly on a refusal-only measure.

> **Read the two blocks separately.** Three of the five metrics are *defined* to be False on a correct abstention — `relevance` and `retrieval_relevance` return False, `groundedness` returns True because a refusal asserts nothing. Pooling all 14 rows into one mean therefore makes good refusal behaviour look like a regression. The evaluation cell prints an answerable block (answer quality) and an unanswerable block (refusal behaviour); `abstention` is the only metric meaningful in both.

Resolution differs between the blocks: 10 answerable rows give steps of 0.1, but 4 unanswerable rows give steps of **0.25**. One flipped question moves that block a quarter of its range, so treat it as directional evidence rather than a measurement.